

TMM4128:

Machine Learning for Engineers

Andrei Lobov

e-mail: andrei.lobov@ntnu.no

phone: +47 45 913 455

room: 213, Verkstedteknisk

web: <http://www.lobov.biz/academia>

Slides:> <http://lobov.biz/academia/mle/250214>

Reference group

Reference groups - quality assurance of education

2-3 representatives for the class. Possible interface to give feedback on different matters related to the course.

Suggested 3 meetings, e.g. i) middle of February; ii) mid of March; and iii) end of April

The group:

- 1) Elisabella Hjelpdahl
- 2) Ana Rey Mata

Looking at yesterday's seminar through RandomForestClassifier

```
tallRatio = self.height / max(self.length, self.width)
longRatio = self.length / max(self.height, self.width)
#flatRatio = 1 / tallRatio
propRatio = max(self.height/self.length, self.height/self.width, self.width/self.length,

    if propRatio < 1.4:
        self.cls = 0 #Proportional
    elif tallRatio >= 1.4:
        self.cls = 1 #Tall
    elif longRatio >= 1.4:
        self.cls = 2 #Long
    else:
        self.cls = 3 #Wide
```

```
from sklearn.metrics import accuracy_score,
confusion_matrix
```

```
y_pred = rf_clf.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
cm = confusion_matrix(y_test, y_pred)
```

```
print(accuracy)
print(cm)
```

```
some_inp = [[1000, 1000, 1000]]
rf_clf.predict(some_inp)
>>>> array([0])
```

More training instances:

```
0.9893
[[ 1624    0    0   66]
 [    0  3322    0   35]
 [    0    0  3305   27]
 [   32   22   32 11535]]
```

0.89

```
[[ 12    1    1   13]
 [  0   27    0    2]
 [  0    0   22    0]
 [  1    2    2  117]]
```

Looking at yesterday's seminar through SGD + more training instances

```
gen = CSVGenerator(100000, PATH, 10, 1000, 10, 1000, 10, 1000)  
gen.generate()
```

```
X = wrkpcs[["Length", "Width", "Height"]]
```

```
y = wrkpcs["Class"]
```

```
X_train, X_test, y_train, y_test = X[:80000], X[80000:], y[:80000], y[80000:]
```

```
y_train_2 = (y_train == 2)
```

```
y_test_2 = (y_test == 2)
```

```
from sklearn.linear_model import SGDClassifier
```

```
sgd_clf = SGDClassifier(max_iter=1000, tol=1e-3, random_state=42)
```

```
sgd_clf.fit(X_train, y_train_2)
```

```
some_inp = [[990, 300, 10]]
```

```
sgd_clf.predict(some_inp)
```

```
>>> array([ True])
```

```
from sklearn.metrics import confusion_matrix  
from sklearn.model_selection import cross_val_predict  
  
y_train_pred = cross_val_predict(sgd_clf, X_train, y_train_2, cv=3)  
confusion_matrix(y_train_2, y_train_pred)  
  
array([[63127, 3487],  
       [ 1837, 11549]])
```

Looking at yesterday's seminar through SVM + more training instances → **Several minutes to train (Fail)**

```
gen = CSVGenerator(100000, PATH, 10, 1000, 10, 1000, 10, 1000)  
gen.generate()
```

```
X = wrkpcs[["Length", "Width", "Height"]]  
y = wrkpcs["Class"]  
X_train, X_test, y_train, y_test = X[:80000], X[80000:], y[:80000], y[80000:]
```

```
from sklearn.svm import SVC  
some_inp = [[1000, 10, 10]]
```

```
svm_clf = SVC(gamma="auto", random_state=42)  
svm_clf.fit(X_train, y_train) # y_train, not y_train_5  
svm_clf.predict(some_inp)  
>>> .... Waiting ...  
>>> Failing : 3
```

Outline : Unsupervised Learning Techniques (code [samples](#))

- Definitions / Motivation
- Clustering
 - K-Means
 - Limits of K-Means
 - Using Clustering for Image Segmentation
 - Using Clustering for Preprocessing
 - Using Clustering for Semi-Supervised Learning
 - DBSCAN
 - Other Clustering Algorithms
- Gaussian Mixtures
 - Anomaly Detection Using Gaussian Mixtures
 - Selecting Number of Clusters
 - Bayesian Gaussian Mixture Models
 - Other Algorithms for Anomaly and Novelty Detection
- Examples

—

Source: Aurelien Geron, “Hands-on machine learning with Scikit-Learn, Keras, and TensorFlow : concepts, tools, and techniques to build intelligent systems”, 2019
([Available online](#)) / **Chapter 9**

Motivation

No labels. Collecting and labelling can easily become an expensive and time consuming task.

How to accommodate changes? “Re-labeling”?

Can the unlabeled data be explored?

Yes, so, unsupervised learning is the exploration of unlabeled data.

Dimensionality reduction is one example of unsupervised learning task.



Definitions

Clustering: group *similar* instances together into **clusters**. Great tool for data analysis, customer segmentation, recommender systems, search engines, image segmentation, semi-supervised learning, dimensionality reduction, and more.

Anomaly detection: learn what “normal” data looks like, and use this to **detect abnormal instances**, such as defective items on a production line or a new trend in a time series.

Density estimation: estimate the ***probability density function (PDF)*** of the random process that generated the dataset. This is commonly used for anomaly detection: instances located in very low-density regions are likely to be anomalies. It is also useful for data analysis and visualization.

Clustering

Steps 1-11

Clustering

Clustering: identifying similar instances and assigning them to *clusters*, i.e., groups of similar instances.

No universal definition of what a cluster is: it's context-dependant. Different algorithms will capture different kinds of clusters. Some look for instances centered around a particular point, called a *centroid*. Others look for continuous regions of densely packed instances: these clusters can take on any shape. Some algorithms are hierarchical, looking for clusters of clusters, etc.

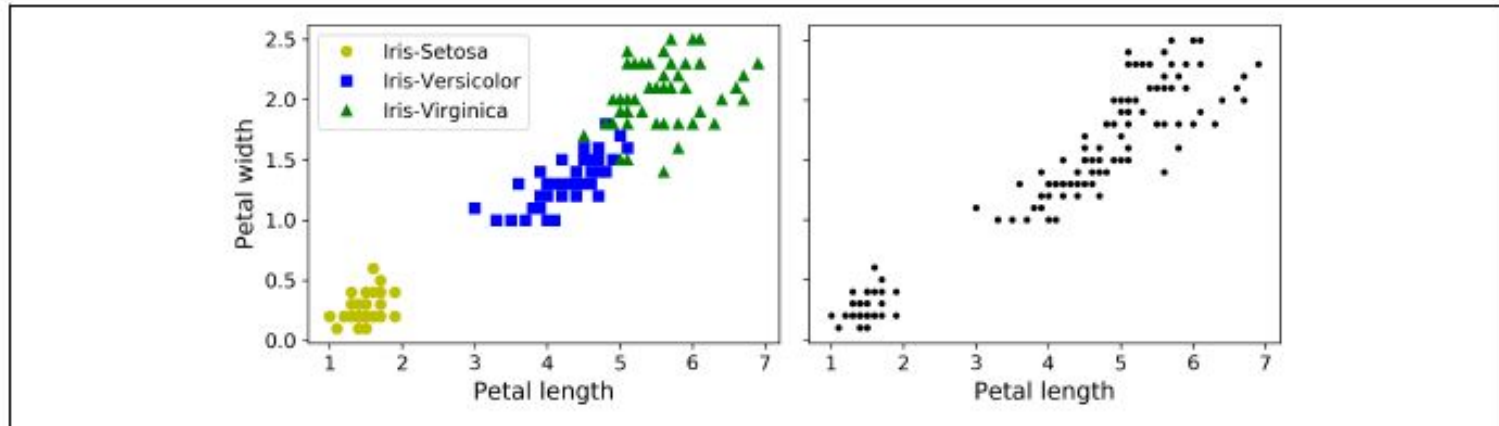


Figure 9-1. Classification (left) versus clustering (right)

Clustering: Applications (1)

Customer segmentation: for example, cluster your customers based on their purchases, their activity on your website, and so on. This is useful to understand who your customers are and what they need, so you can adapt your products and marketing campaigns to each segment. This can be useful in recommender systems to suggest content that other users in the same cluster enjoyed.

Data analysis: when analyzing a new dataset, it is often useful to first discover clusters of similar instances, as it is often easier to analyze clusters separately.

Clustering: Applications (2)

Dimensionality reduction technique: once a dataset has been clustered, it is usually possible to measure each instance's affinity with each cluster (affinity is any measure of how well an instance fits into a cluster). Each instance's feature vector $\mathbf{x}$ can then be replaced with the vector of its cluster affinities. If there are k clusters, then this vector is k dimensional. This is typically much lower dimensional than the original feature vector, but it can preserve enough information for further processing.

Clustering: Applications (3)

Dimensionality reduction technique: once a dataset has been clustered, it is usually possible to measure each instance's affinity with each cluster (affinity is any measure of how well an instance fits into a cluster). Each instance's feature vector $\mathbf{x}$ can then be replaced with the vector of its cluster affinities. If there are k clusters, then this vector is k dimensional. This is typically much lower dimensional than the original feature vector, but it can preserve enough information for further processing.

Clustering: Applications (4)

Anomaly detection (also called outlier detection): **any instance that has a low affinity to all the clusters is likely to be an anomaly**. For example, if you have clustered the users of your website based on their behavior, you can detect users with unusual behavior, such as an unusual number of requests per second, and so on. Anomaly detection is particularly useful in detecting defects in manufacturing, or for fraud detection.

Semi-supervised learning: if you only have **a few labels**, you could perform clustering and **propagate the labels to all the instances** in the same cluster. This can greatly increase the amount of labels available for a subsequent supervised learning algorithm, and thus improve its performance.

Clustering: Applications (5)

Search engines: for example, some search engines let you search for images that are similar to a reference image. To build such a system, you would first apply a clustering algorithm to all the images in your database: similar images would end up in the same cluster. Then when a user provides a reference image, all you need to do is to find this image cluster using the trained clustering model, and you can then simply return all the images from this cluster.

Segment an image: by clustering pixels according to their color, then replacing each pixel's color with the mean color of its cluster, it is possible to reduce the number of different colors in the image considerably. This technique is used in many object detection and tracking systems, as it makes it easier to detect the contour of each object.

K-Means

Steps 12-69

K-Means

```
from sklearn.cluster import KMeans
k = 5
kmeans = KMeans(n_clusters=k)
y_pred = kmeans.fit_predict(X)
```

```
>>> y_pred
array([4, 0, 1, ..., 2, 1, 0], dtype=int32)
>>> y_pred is kmeans.labels_
True
```

```
>>> kmeans.cluster_centers_
array([[ -2.80389616,  1.80117999],
       [  0.20876306,  2.25551336],
       [-2.79290307,  2.79641063],
       [-1.46679593,  2.28585348],
       [-2.80037642,  1.30082566]])
```

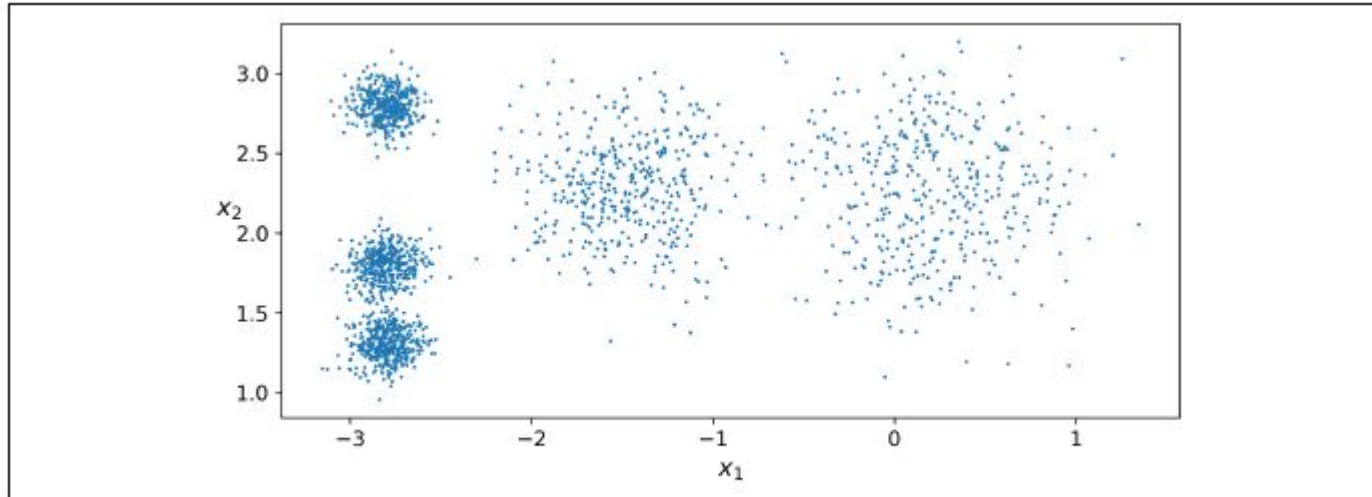


Figure 9-2. An unlabeled dataset composed of five blobs of instances

K-Means : Predicting and decision boundaries

```
>>> X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])  
>>> kmeans.predict(X_new)  
array([1, 1, 2, 2], dtype=int32)
```

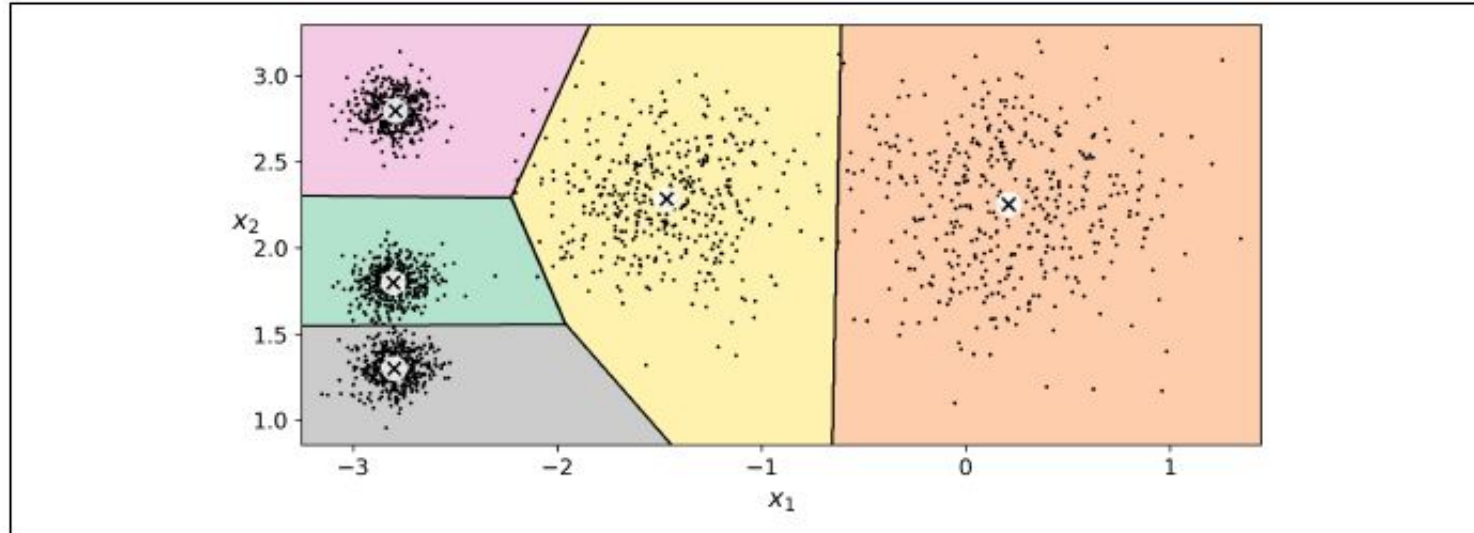


Figure 9-3. K-Means decision boundaries (Voronoi tessellation)

K-Means : Hard and soft clustering

Hard clustering : Assigning each instance to a **single cluster**.

Soft clustering : Each instance gets a score per cluster.

For example, the score can be the distance between the instance and the centroid, or conversely it can be a similarity score (or affinity) such as the Gaussian Radial Basis Function.

For the soft KMeans.transform(inpVal) instead of predict().

K-Means : Measuring distance from every instance to every centroid

```
X_new = np.array([[0, 2], [3, 2], [-3, 3], [-3, 2.5]])  
>>> kmeans.transform(X_new)  
array([[2.81093633, 0.32995317, 2.9042344 , 1.49439034, 2.88633901],  
       [5.80730058, 2.80290755, 5.84739223, 4.4759332 , 5.84236351],  
       [1.21475352, 3.29399768, 0.29040966, 1.69136631, 1.71086031],  
       [0.72581411, 3.21806371, 0.36159148, 1.54808703, 1.21567622]])
```

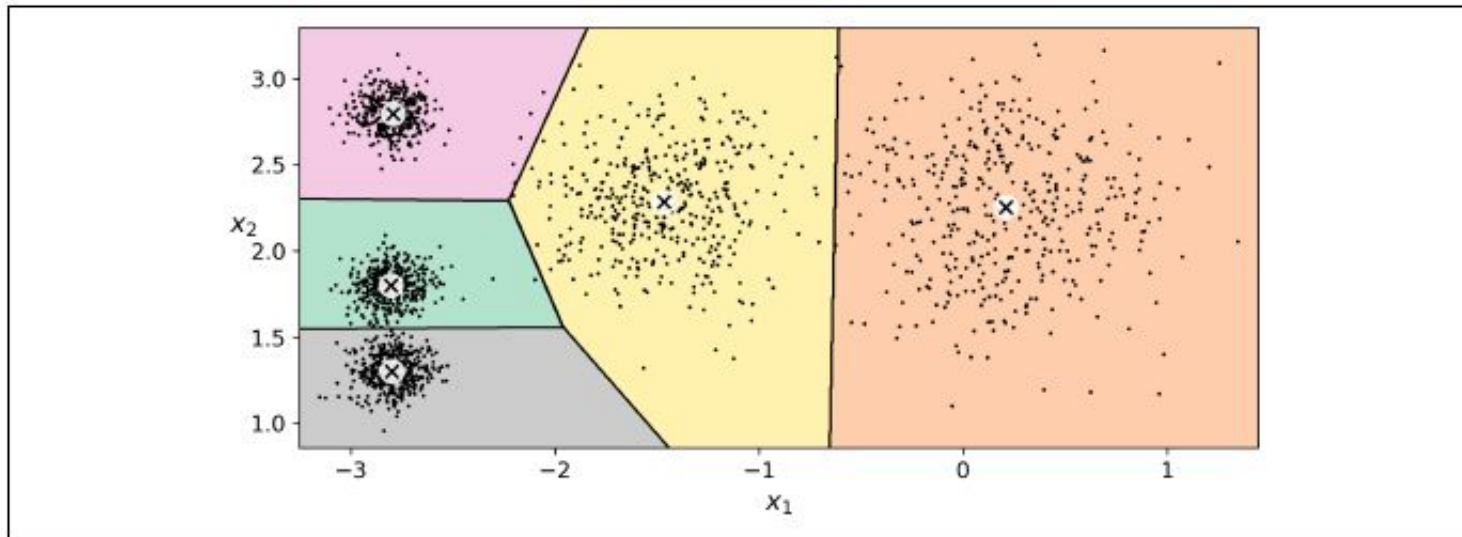


Figure 9-3. K-Means decision boundaries (Voronoi tessellation)

K-Means : Algorithm (1)

- Start by placing the centroids randomly (e.g., by picking k instances at random and using their locations as centroids).
- Then label the instances, update the centroids, label the instances, update the centroids, and so on until the centroids stop moving.
- The algorithm is guaranteed to converge in a finite number of steps (usually quite small), it will not oscillate forever.

K-Means : Algorithm (2)

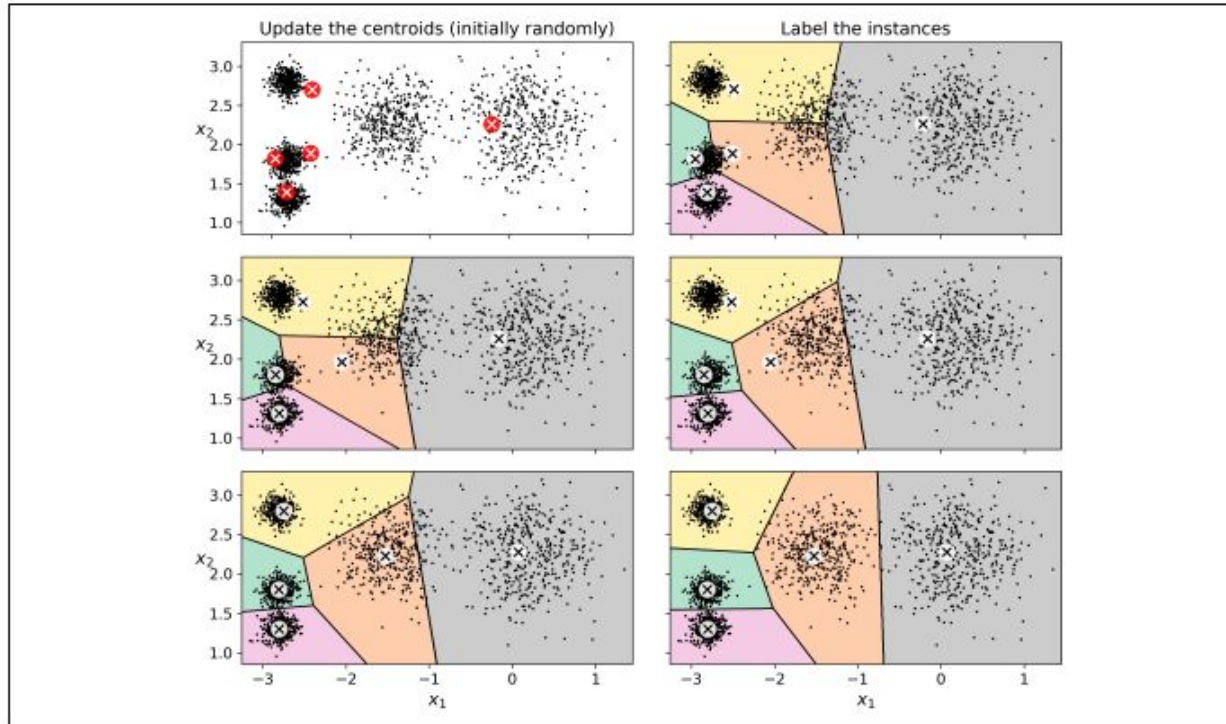


Figure 9-4. The K-Means algorithm

K-Means : Algorithm (2)

Although the algorithm is guaranteed to converge, it may not converge to the right solution (i.e., it may converge to a local optimum): this depends on the centroid initialization.

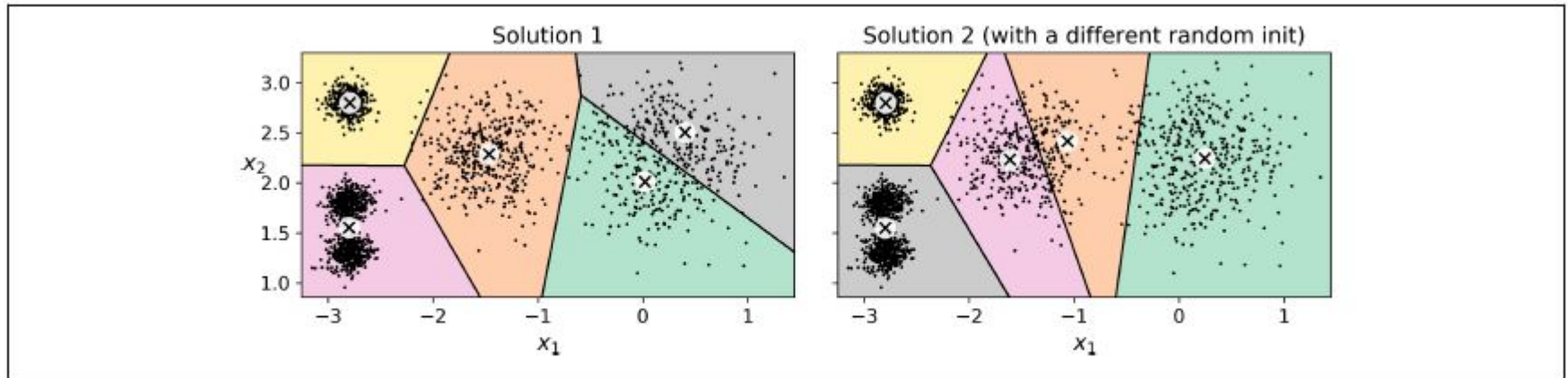


Figure 9-5. Sub-optimal solutions due to unlucky centroid initializations

Next: How to mitigate that risk?

Centroid Initialization Methods

```
good_init = np.array([[ -3,  3], [ -3,  2], [ -3,  1], [ -1,  2], [  0,  2]])  
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1)
```

The KMeans class runs the algorithm **n_init** times and keeps the model with the lowest inertia: in the example, the model in Figure 9-3 will be selected (unless we are very unlucky with n_init consecutive random.

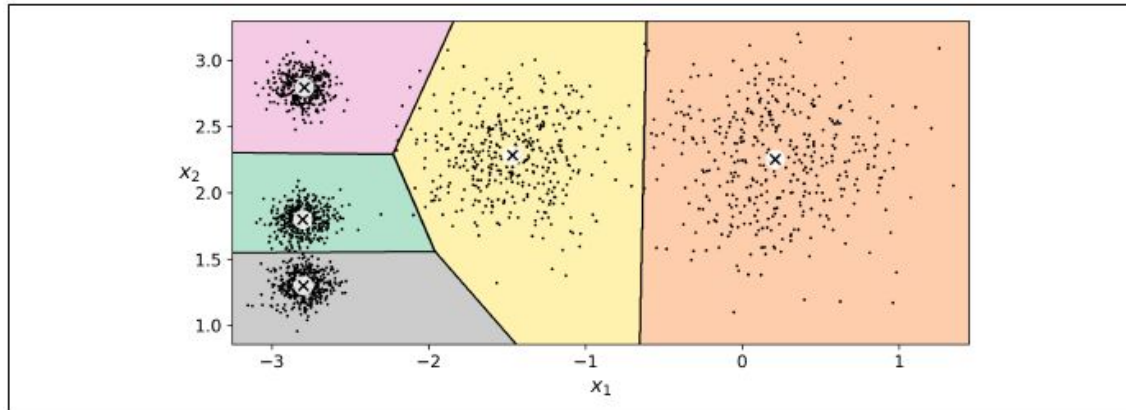


Figure 9-3. K-Means decision boundaries (Voronoi tessellation)

Centroid Initialization Methods : Inertia

```
good_init = np.array([[ -3,  3], [ -3,  2], [ -3,  1], [ -1,  2], [ 0,  2]])
kmeans = KMeans(n_clusters=5, init=good_init, n_init=1)
```

by default, **n_init** is equal to 10, so the whole algorithm runs 10 times on the calling of **fit()**.

How the best solution is selected? Performance metrics - **inertia**: the mean squared distance between each instance and its closest centroid.

```
>>> kmeans.inertia_
211.59853725816856
```

Also, the score (Negative in this case, for the “great(er) is better” rule).

```
>>> kmeans.score(X)
-211.59853725816856
```

KMeans++

- Take one centroid $c^{(1)}$, chosen uniformly at random from the dataset.
 - Take a new centroid $c^{(i)}$, choosing an instance $x^{(i)}$ with probability:
 $D(x^{(i)})^2 / \sum_{j=1}^m D(x^{(j)})^2$, where $D(x^{(i)})$ the distance between the instance $x^{(i)}$ and the closest centroid that was already chosen.
 - This probability distribution ensures that **instances further away from already chosen centroids are much more likely be selected as centroids.**
 - Repeat the previous step until all k centroids have been chosen.
-
- The KMeans class actually uses this initialization method by default.
Set **init**="random" for the original algorithm.

Accelerated K-Means and Mini-batch K-Means (1)

Using triangle inequality (straight line is always the shortest).

Web-scale k-mean clustering : Instead of using the full dataset at each iteration, the algorithm is capable of using mini-batches, moving the centroids just slightly at each iteration. Possible to cluster huge datasets that do not fit in memory.

```
from sklearn.cluster import MiniBatchKMeans  
  
minibatch_kmeans = MiniBatchKMeans(n_clusters=5)  
minibatch_kmeans.fit(X)
```

Accelerated K-Means and Mini-batch K-Means (2)

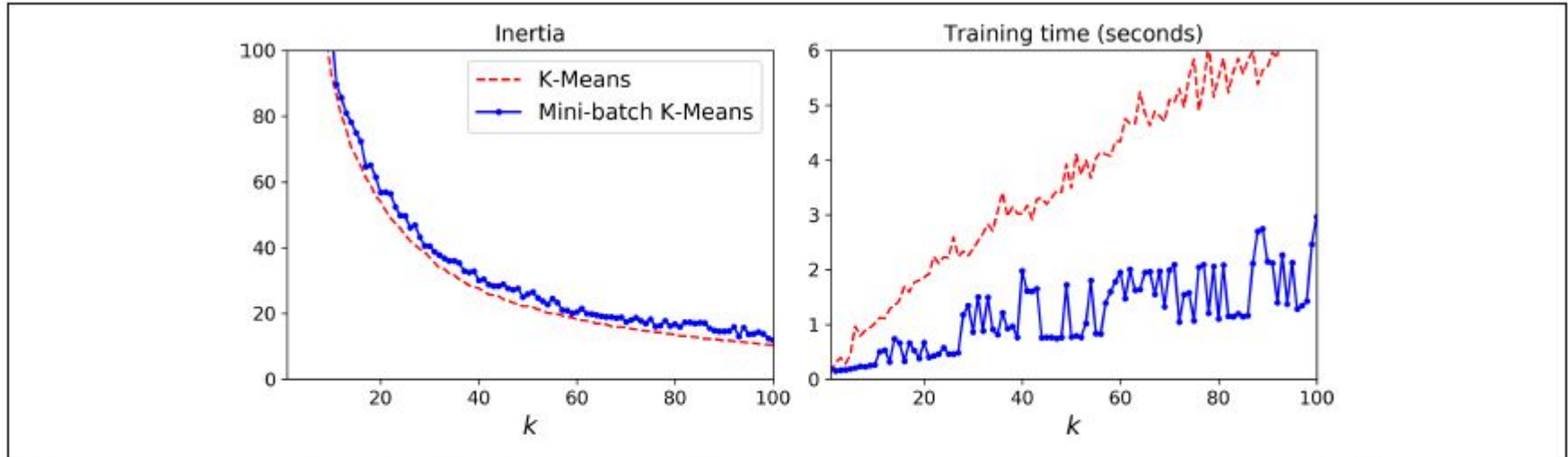


Figure 9-6. Mini-batch K-Means vs K-Means: worse inertia as k increases (left) but much faster (right)

Finding the Optimal Number of Clusters (1)

So, how to decide how many clusters to use (if there are no good or obvious hints)?

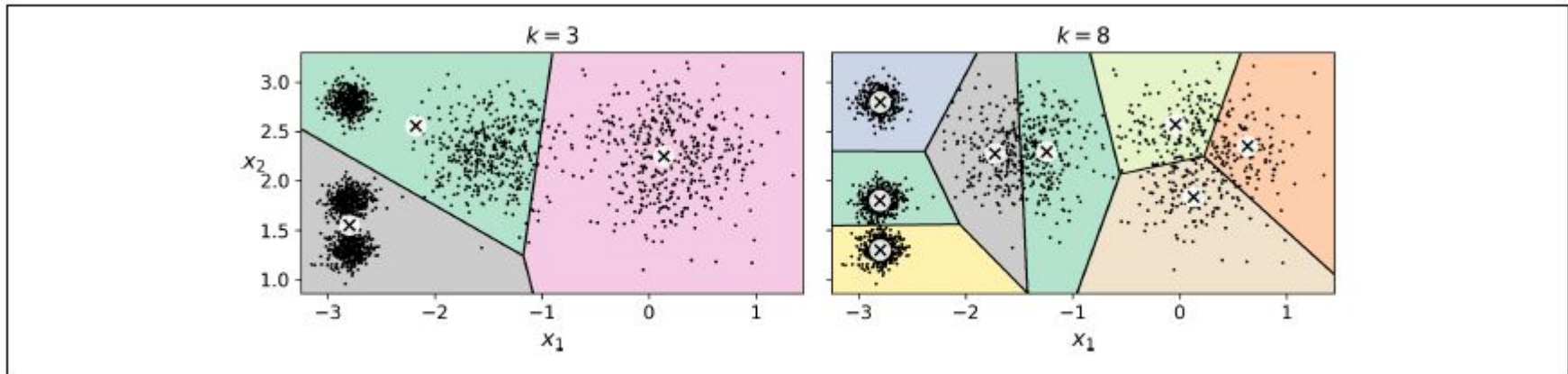


Figure 9-7. Bad choices for the number of clusters

Finding the Optimal Number of Clusters (2)

Picking the model with the lowest inertia?

Unfortunately, it is not that simple. The inertia for $k=3$ is 653.2, which is much higher than for $k=5$ (which was 211.6).

But with $k=8$, the inertia is just 119.1. **The inertia is not a good performance metric when trying to choose k since it keeps getting lower as we increase k .** The more clusters there are, the closer each instance will be to its closest centroid

Finding the Optimal Number of Clusters (3)

Elbow rule : $k = 4$

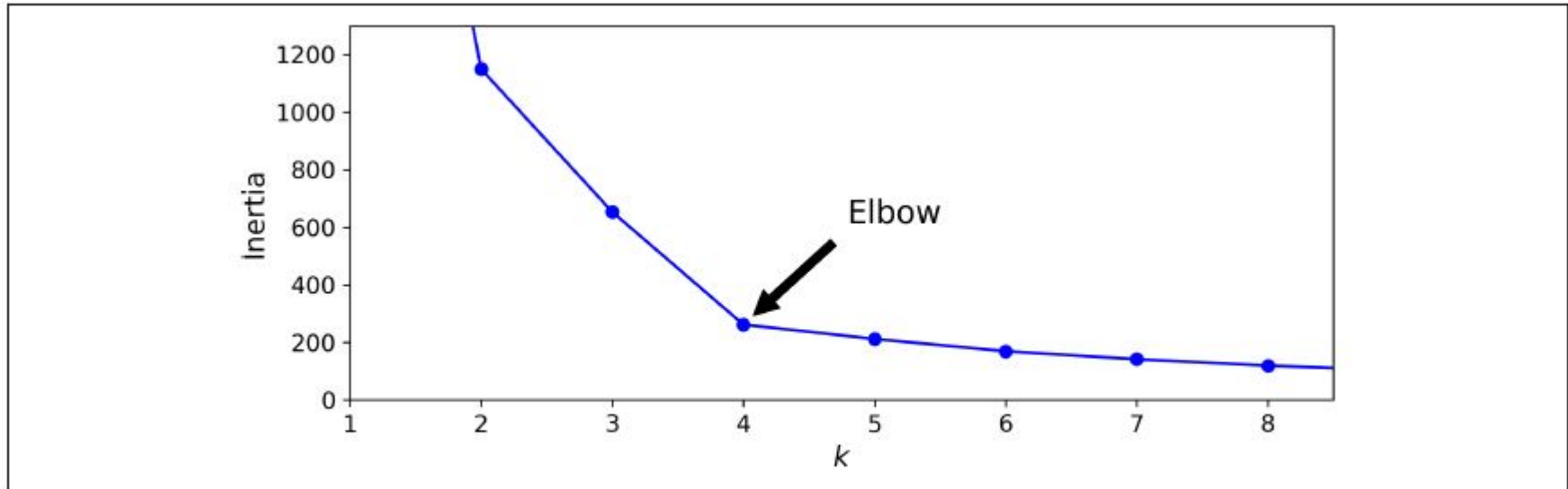


Figure 9-8. Selecting the number of clusters k using the “elbow rule”

Finding the Optimal Number of Clusters (4)

Silhouette score : silhouette coefficient within $[-1, +1]$

~ +1 : instance well inside its own cluster

~ -1 : instance may have been assigned to the wrong cluster.

```
>>> from sklearn.metrics import silhouette_score  
>>> silhouette_score(X, kmeans.labels_)  
0.655517642572828
```

Finding the Optimal Number of Clusters (5)

Silhouette score : silhouette coefficient within $[-1, +1]$

$\sim +1$: instance well inside its own cluster

~ -1 : instance may have been assigned to the wrong cluster.

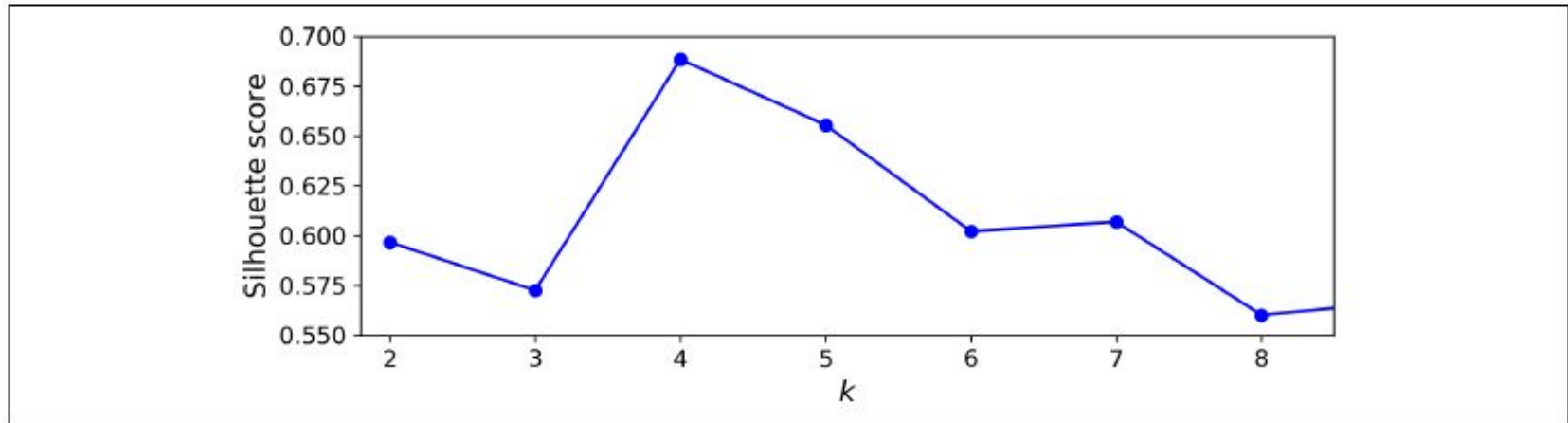


Figure 9-9. Selecting the number of clusters k using the silhouette score

Finding the Optimal Number of Clusters (6)

Silhouette diagram. So, $k=5$ seems like a good idea.

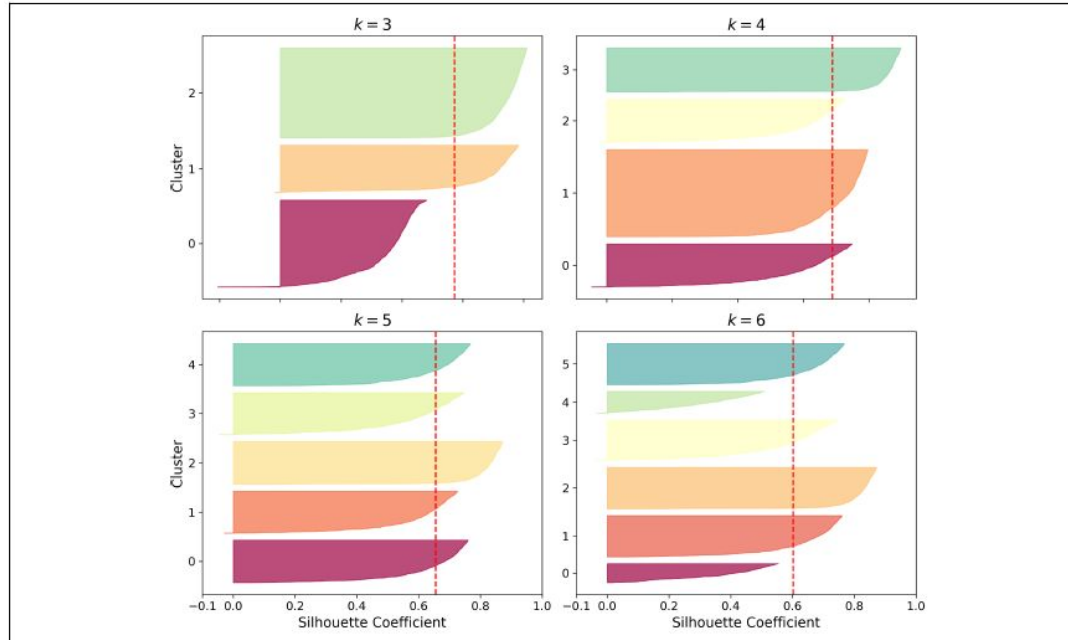


Figure 9-10. Silhouette analysis: comparing the silhouette diagrams for various values of k

Limits of K-Means

Steps 70-73

Limits

- K-Means is not perfect
 - Needs to run several times to avoid sub-optimal solutions.
 - One needs to specify the number of clusters.
 - Does not behave well for the clusters of varying sizes, densities, or non-spherical shapes.

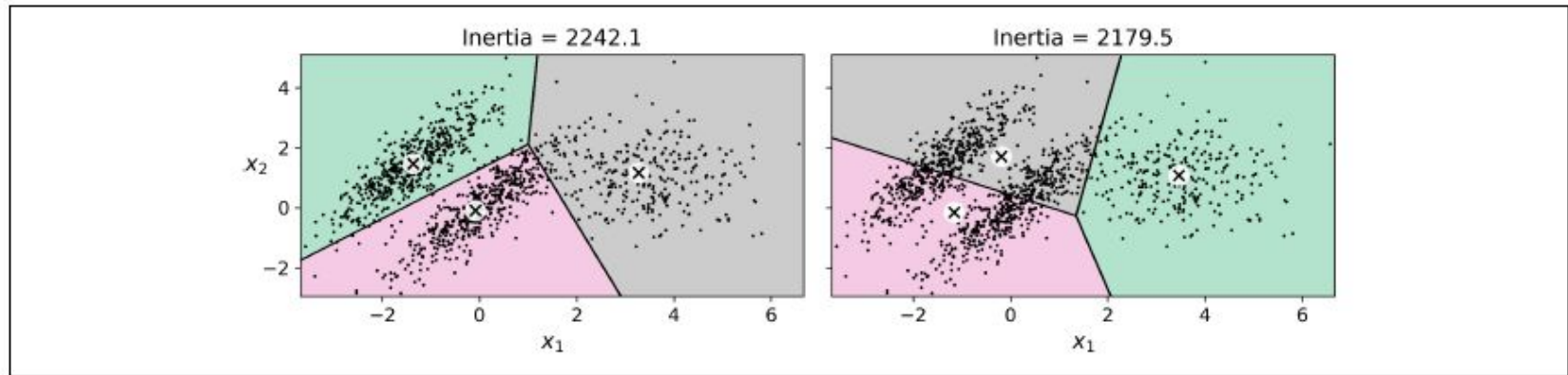


Figure 9-11. K-Means fails to cluster these ellipsoidal blobs properly

On scaling

- Important to scale the input features before you run K-Means, or else the clusters may be very stretched, and K-Means will perform poorly.
- Scaling the features does not guarantee that all the clusters will be nice and spherical, but it generally improves the performance.

Takeaways

- Clustering and variety of applications. A tool to learn about the data.
- K-Means principles, evaluation and selection of k - the number of clusters.
- Being aware of limitations.